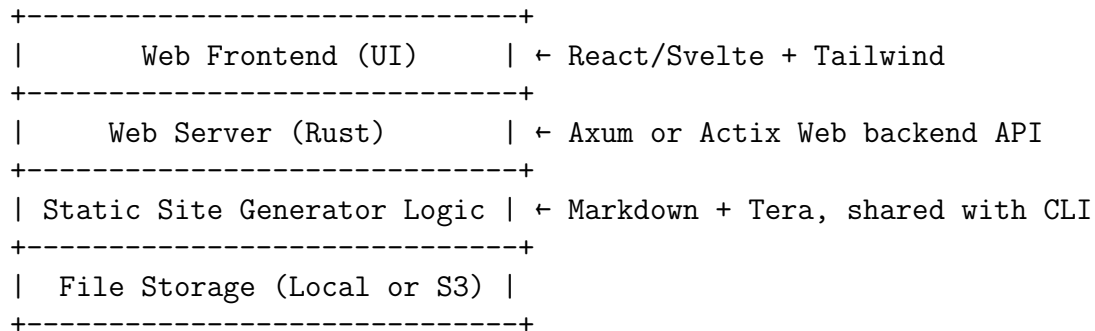


Web-Based GUI for Static Site Generator in Rust

Overview

This document outlines the architecture and implementation plan for a web-based GUI version of a Static Site Generator built in Rust. The system allows users to edit content, preview the site, analyze SEO, and deploy — all from their browser.

Architecture Overview



Technology Stack

- **Frontend:** React, Svelte, or Vue + Tailwind CSS
- **Backend:** Axum or Actix-web (Rust)
- **Templating:** Tera
- **Markdown:** pulldown-cmark
- **Deployment:** GitHub Pages, Netlify API
- **Optional:** JWT-based Auth, S3 storage, SQLite

Web App Features

1. Markdown Editor

- Live preview with syntax highlighting
- Save as draft or publish

2. Theme Switcher

- Select from Tera templates
- Preview updates instantly

3. SEO Analyzer

- Checks for: title, meta description, Open Graph, Twitter card, alt tags, JSON-LD
- Shows SEO score and recommendations

4. Build + Preview

- One-click static build
- Preview site in iframe

5. Deploy Options

- Deploy to GitHub Pages via token
- Upload to Netlify via API
- Export as ZIP archive

Project Structure

project-root/

```
frontend/           # React/Svelte app
  public/
  src/
  vite.config.js
```

```
backend/           # Rust Axum API
  src/
    main.rs
    routes.rs
    api/
    ssg/
    utils.rs
  Cargo.toml
```

```
templates/         # Tera themes
content/           # Markdown pages
static/            # Assets
public/            # Output
config.toml
```

Workflow Overview

1. User visits website
2. Writes content or chooses demo template
3. Clicks “Build” → API call triggers SSG

4. HTML output stored in `public/`
5. Preview shown in `iframe`
6. User downloads ZIP or deploys

Deployment Options

- Host full stack on Render, Fly.io, or Vercel + Supabase
- Static demo sites can be served from Cloudflare Pages
- User subdomains: `username.yourapp.com`

Action Plan

1. Refactor CLI SSG into a reusable Rust library
2. Create Axum-based API server with endpoints:
 - `/build`
 - `/analyze`
 - `/deploy`
3. Build frontend UI in React or Svelte
4. Connect API to build/preview logic
5. Add ZIP export and deploy integration
6. Launch demo version

Optional Enhancements

- User authentication (JWT/OAuth)
- Project saving and loading
- Template marketplace
- Usage analytics dashboard